



WEBD 1000 — Website Development

Workshop 0 — Tooling & Workflow Readiness

Course: WEBD 1000 — Website Development **Activity:** Workshop 0 **Type:** Required / Formative **Topic:** Topic 1 — Course Tooling & Web Foundations **Relevant Learning Outcome:** Outcome 1 — *Evaluate a variety of web sites for usability and accessibility.*

Activity Overview

Welcome to **WEBD 1000 — Website Development**.

Before you begin evaluating, planning, designing, and developing websites, you need a working development environment and an understanding of the workflow you'll use throughout the course.

Web development involves more than writing HTML and CSS. Developers work with source-code editors, browsers, development servers, version-control systems, remote repositories, testing tools, and web-hosting platforms. In this workshop, you'll begin working with these technologies as parts of one connected development process.

You will establish your WEBD 1000 development environment using:

- **Visual Studio Code (VS Code)** as your development environment;
- **Live Server** for viewing and testing websites while you develop them;
- **Git** for tracking changes to your project;
- **GitHub** for storing your project repository online;
- **GitHub Pages** for publishing your website to the Web;
- **Browser Developer Tools** for inspecting and troubleshooting webpages.

You'll configure these tools, create a small web project, test it locally, store it in a GitHub repository, and publish it using GitHub Pages.

The workflow you establish here will become part of your normal development practice throughout WEBD 1000.

The Big Picture

By the end of this workshop, you'll have implemented the following workflow:



This workshop is **formative**. The goal isn't mastery of every technology today. The goal is to make sure your environment works and that you understand how the pieces fit together before later workshops and assessments depend on them.

Learning Objectives

By completing this workshop, you will be able to:

1. Explain the basic roles of VS Code, Live Server, Git, GitHub, GitHub Pages, a web browser, and browser Developer Tools.
2. Configure a web-development environment on Windows, macOS, or Linux.
3. Create and organize a local web-development project.
4. Create and edit a basic HTML document using VS Code.
5. Use Live Server to run and view a website during development.
6. Use browser Developer Tools to inspect a webpage.
7. Create or configure a GitHub account for course development.
8. Create a GitHub repository for WEBD 1000.

9. Connect a local project to a GitHub repository.
 10. Use the basic Git workflow to commit and push project changes.
 11. Publish a website using GitHub Pages.
 12. Distinguish between local development, version control, remote source-code storage, and web publishing.
 13. Explain how this workflow will support your Semester Website Project.
-

Concept Review / Lesson Content

1. What Is Web Development?

When you visit a website, your browser presents a visual interface containing text, images, navigation, buttons, forms, and other content.

Behind that interface are files and technologies that describe what the browser should display and how it should present that information.

Two technologies you'll work with extensively in WEBD 1000 are:

HTML — HyperText Markup Language

HTML describes the **structure and meaning** of webpage content.

HTML can describe elements such as:

- headings;
- paragraphs;
- navigation;
- images;
- links;
- sections;
- headers;
- footers.

CSS — Cascading Style Sheets

CSS controls the **presentation and layout** of HTML content.

CSS can control:

- colours;
- typography;
- spacing;
- borders;
- alignment;
- page layouts;
- responsive behaviour.

You'll learn HTML and CSS progressively throughout this course. For Workshop 0, the immediate goal is to establish the environment in which you'll develop them.

2. Visual Studio Code

Visual Studio Code, commonly called **VS Code**, is a source-code editor.

It's available for:

- Windows;
- macOS;
- Linux.

That matters because web technologies aren't restricted to one operating system. A student using Windows and a student using macOS may have different operating-system interfaces, but both can create the same HTML and CSS and follow essentially the same development workflow.

Throughout WEBD 1000, VS Code will act as your primary development workspace.

You'll use it to:

- create files;
- edit source code;
- organize projects;
- install development extensions;
- access Git;
- interact with GitHub;
- troubleshoot projects.

Think of VS Code as the place where most of your development work begins.

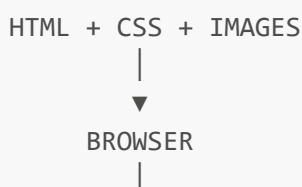
3. The Web Browser

A browser isn't simply a tool for visiting websites.

For a web developer, the browser is also a **development and testing environment**.

Browsers interpret web resources and convert them into the visual pages users see.

Conceptually:



▼
RENDERED WEBPAGE

During this course, you'll repeatedly move between your source code and the browser. You'll make a change in VS Code, view the result in the browser, inspect what happened, and return to the code to continue developing or correct a problem.

4. Browser Developer Tools

Modern browsers contain built-in **Developer Tools**, commonly called **DevTools**.

DevTools let you inspect what the browser is actually rendering.

You'll eventually use DevTools to investigate:

- HTML;
- CSS;
- page layouts;
- responsive behaviour;
- accessibility;
- errors;
- network activity;
- browser behaviour.

This means the browser becomes part of your normal troubleshooting workflow rather than simply the place where you look at the finished product.

5. Live Server

During development, you need a convenient way to run your website locally.

A VS Code extension such as **Live Server** provides a lightweight local development web server.

Instead of repeatedly opening an HTML file manually, Live Server can make your project available through an address similar to:

```
http://127.0.0.1:5500/
```

or:

```
http://localhost:5500/
```

When you save changes, the browser can refresh to display the updated page.

This gives you an efficient development loop:

```
EDIT
↓
SAVE
↓
VIEW
↓
INSPECT
↓
CORRECT
↓
REPEAT
```

Important

Live Server is a **local development server**.

It helps you develop and test the website on your computer.

It does **not** publish your website to the Internet. Later in this workshop, you'll see the difference when you use GitHub Pages.

6. Git

As projects become larger, simply saving files isn't enough.

Developers need a way to track meaningful changes over time.

Git is a distributed version-control system that records changes to project files. Git allows you to create checkpoints called **commits**.

For example:

```
Create project structure
↓
COMMIT
↓
Add navigation
↓
COMMIT
↓
Add page content
↓
COMMIT
↓
```

Implement responsive layout



COMMIT

A Git history therefore tells part of the story of how a project developed.

This will become especially useful for your Semester Website Project because the website won't be created all at once. It will develop progressively throughout the semester.

7. GitHub

Git and GitHub are related, but they are **not the same thing**.

Git is the version-control technology.

GitHub is an online service capable of hosting Git repositories.

A simple mental model is:

YOUR COMPUTER

Local Git Repository



| PUSH



GITHUB

Remote Git Repository

Having your project on GitHub provides several benefits.

Your work can:

- exist somewhere other than your individual computer;
- maintain a development history;
- be accessed from other environments;
- be reviewed by your instructor where appropriate;
- support publishing through GitHub Pages;
- contribute to your developing technical portfolio.

Throughout the course, you'll become increasingly comfortable with the relationship between the work on your computer and the repository stored on GitHub.

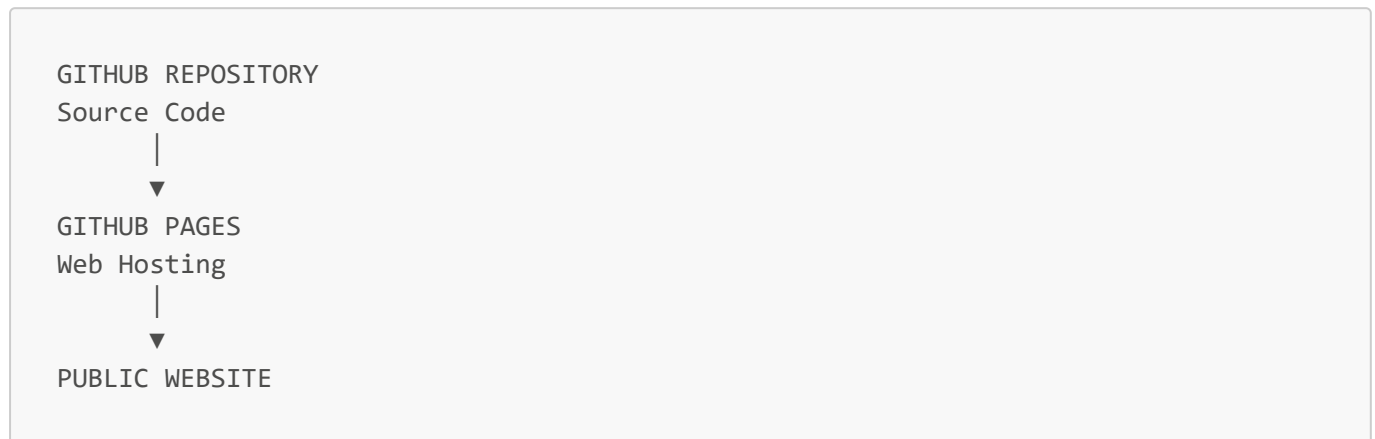
8. GitHub Pages

A GitHub repository contains your **source files**.

That doesn't automatically mean you have a published website.

GitHub Pages provides web hosting suitable for static websites built with technologies such as HTML and CSS.

Conceptually:



This distinction is important.

GitHub Repository

Stores and tracks the project's source files.

GitHub Pages

Makes appropriate web content available through a web address.

Throughout WEBD 1000, GitHub Pages gives your Semester Website Project a way to exist **outside the NSCC learning environment**.

Your instructor can view the evolving project through a browser, and you can experience your work as an actual published website rather than only as files submitted to a course. As the project develops, it can also begin contributing to a body of technical work that exists beyond an individual assignment.

Remember: Content published using GitHub Pages may be publicly accessible. Never place passwords, authentication credentials, API keys, confidential information, private personal information, or other sensitive material in a public repository or website.

Integrated Tutorial / Guided Activities

Activity A — Prepare Your Development Environment

Task A1 — Install Visual Studio Code

Install Visual Studio Code for your operating system if it isn't already installed.

Choose the appropriate installer for:

- Windows;
- macOS; or
- Linux.

Launch VS Code after installation.

Explore the interface and locate:

- **Explorer**
- **Search**
- **Source Control**
- **Extensions**
- **Editor**
- **Terminal**

You aren't expected to understand every feature yet. For now, become familiar with where these areas are located.

Verification Checkpoint A

Before continuing, verify:

- ☐ VS Code is installed.
- ☐ VS Code opens successfully.
- ☐ I can locate Explorer.
- ☐ I can locate Source Control.
- ☐ I can locate Extensions.
- ☐ I can open the integrated Terminal.

Activity B — Install Live Server

Task B1 — Open Extensions

Open the **Extensions** view in VS Code.

Search for the Live Server extension identified by your instructor.

Before installing an extension, examine:

- its name;
- publisher;
- description;
- installation count/reputation where available.

Install the extension.

Why Are We Doing This?

Extensions add functionality to VS Code.

Live Server will provide the local web-development environment you'll use to preview your pages while you work.

At this stage, remember the distinction:

LIVE SERVER
Used while developing locally

versus

GITHUB PAGES
Used to publish the website

Verification Checkpoint B

- ☐ Live Server is installed.
- ☐ VS Code recognizes the extension.
- ☐ I understand that Live Server is used for local development.
- ☐ I understand that Live Server does not publish my site to the Internet.

Activity C — Prepare Git

Task C1 — Check for Git

Open the VS Code Terminal.

Run:

```
git --version
```

If Git is installed, you should receive a response showing the installed version.

If the command isn't recognized, follow the installation instructions demonstrated by your instructor for your operating system.

After installation, restart VS Code if required and run the command again.

💡 Try This

Run:

```
git --help
```

You don't need to understand the available commands yet.

What does this tell you about Git?

Task C2 — Configure Your Identity

Git associates commits with an identity.

Follow your instructor's directions to configure your development identity.

For example:

```
git config --global user.name "Your Name"
```

and:

```
git config --global user.email "your-email@example.com"
```

Use an appropriate email address for your GitHub/development environment.

Verification Checkpoint C

- ☐ `git --version` works.
 - ☐ Git is available from the VS Code Terminal.
 - ☐ my Git identity is configured.
 - ☐ VS Code can access Git functionality.
-

Activity D — Create Your GitHub Account

Task D1 — Sign In or Register

Open GitHub.

If you already have an account, sign in.

If you don't have an account, create one.

Choosing a Username

Think carefully about your username.

GitHub is widely used by software developers and IT professionals. Material you intentionally make public may eventually be seen by:

- classmates;
- instructors;

- employers;
- clients;
- other developers.

Choose a username you'd be comfortable using professionally.

Verification Checkpoint D

- ☐ I can sign in to GitHub.
- ☐ I know my GitHub username.
- ☐ I understand that some GitHub content may be publicly visible.

Activity E — Create the WEBD 1000 Repository

Task E1 — Create the Repository

Create a repository for this course.

The recommended repository name is:

WEBD1000

Your instructor may specify another naming convention if required.

This repository will give you an organized location for your WEBD 1000 development work.

As the semester progresses, it might eventually resemble:

```
WEBD1000/  
|  
├── workshop-0/  
├── workshop-1/  
├── workshop-2/  
|  
├── assignments/  
└── semester-project/
```

Don't create all of these directories unless directed to do so.

The example is intended to show how one repository can support an organized semester of development.

Verification Checkpoint E

- ☐ My WEBD1000 repository exists.
- ☐ I can open it through GitHub.

- ☐ I understand that this is a Git repository hosted remotely on GitHub.

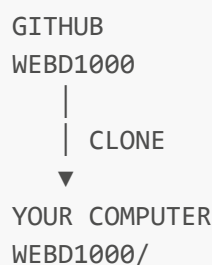
Activity F — Connect GitHub and VS Code

Your GitHub repository now exists remotely.

Next, you need a local copy where you can perform your development work.

Your instructor will demonstrate the course workflow for **cloning** the repository.

Conceptually:



Open the resulting **WEBD1000** directory in VS Code.

What Just Happened?

You now have two related locations:



Changes don't automatically move between them. Git provides the workflow you'll use to manage those changes.

Verification Checkpoint F

- ☐ The repository exists on my computer.
- ☐ I opened the repository folder in VS Code.
- ☐ VS Code recognizes the Git repository.
- ☐ I can see the repository in Source Control.
- ☐ I understand the basic difference between the local and remote repository.

Activity G — Create Your First Web Project

Inside your repository, create:

```
workshop-0/
```

Inside `workshop-0`, create:

```
workshop-0/  
├── index.html  
├── css/  
│   └── style.css  
└── images/
```

This introduces an important web-development practice:

Projects should have an intentional and predictable directory structure.

Task G1 — Create `index.html`

Add the following starter document:

```
<!DOCTYPE html>  
<html lang="en">  
  
  <head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>WEBD 1000 – Workshop 0</title>  
  </head>  
  
  <body>  
  
    <header>  
      <h1>WEBD 1000</h1>  
    </header>  
  
    <main>  
      <h2>My Web Development Environment</h2>  
  
      <p>  
        My WEBD 1000 development environment is working.  
      </p>  
    </main>  
  </body>  
</html>
```

```
</p>

<p>
  This page was created using Visual Studio Code.
</p>
</main>

<footer>
  <p>Workshop 0 – Tooling & Workflow Readiness</p>
</footer>

</body>

</html>
```

Save the document.

Important

You are **not expected to understand or memorize all of this HTML yet**.

HTML will be taught systematically later in the course. For now, look at the structure and see what you can infer from the element names.

Try This

Without researching the answer, identify what you think these elements might represent:

```
<header>
<main>
<footer>
<h1>
<p>
```

Use the names of the elements as clues.

Activity H — Run the Website with Live Server

Locate `index.html` in VS Code.

Launch the page using Live Server.

Your browser should open.

Examine the address. It may resemble:

```
http://127.0.0.1:5500/workshop-0/
```

or:

```
http://localhost:5500/workshop-0/
```

Notice that this doesn't look like a normal public website address.

That's because Live Server is running the website locally on your computer.

Task H1 — Modify the Page

Return to VS Code.

Change:

```
<h2>My Web Development Environment</h2>
```

to a heading of your choice.

Add another paragraph describing one thing you'd like to learn about web development.

Save the document.

Return to your browser and observe what happens.

You've now completed the fundamental development cycle:

EDIT → SAVE → VIEW

Repeat the process at least once.

Try This — Break Something

Temporarily change:

```
<h1>WEBD 1000</h1>
```

to:

```
<h1>WEBD 1000
```

Save the page and observe what happens.

Then restore the correct code.

One of the most important skills you'll develop this semester is learning to:

make a change → **observe the result** → **identify the problem** → **correct it**

Verification Checkpoint H

- ☐ My webpage runs through Live Server.
- ☐ I can identify the local URL.
- ☐ I can change HTML in VS Code.
- ☐ I can save the document.
- ☐ I can see the updated result in the browser.
- ☐ I understand that this version is running locally.

Activity I — Explore Browser Developer Tools

With your page running, open the browser's Developer Tools.

Locate the **Elements** or **Inspector** view.

Find:

```
<header>
<main>
<footer>
<h1>
<h2>
<p>
```

Use the element-selection tool to select content directly from your webpage.

Observe how the browser identifies the corresponding HTML.

Try This — Temporary Browser Changes

Using Developer Tools, temporarily modify the text of your `<h1>`.

Do **not** change the source file in VS Code.

Refresh the browser.

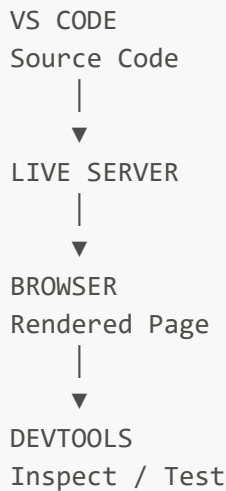
What Happened?

The change disappears.

Why?

Because Developer Tools modified the browser's current representation of the page. It didn't modify your actual source file.

This gives us another useful mental model:



Verification Checkpoint I

- ☐ I can open Developer Tools.
- ☐ I can inspect an HTML element.
- ☐ I can relate rendered content to its HTML source.
- ☐ I understand that temporary DevTools changes do not automatically modify my source files.

Activity J — Create Your First Commit

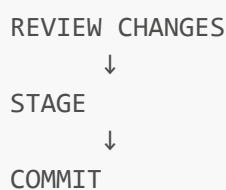
You now have meaningful project files.

It's time to record a checkpoint using Git.

Open **Source Control** in VS Code.

Observe the files Git identifies as new or changed.

Your instructor will demonstrate the basic process:



Use a meaningful commit message such as:

```
Complete Workshop 0 website setup
```

Avoid messages such as:

```
stuff
```

```
changes
```

```
asdf
```

A useful commit message should help someone understand **what changed** without having to inspect every file.

Verification Checkpoint J

- ☐ Git recognizes my project changes.
- ☐ I reviewed the files being committed.
- ☐ I created a commit.
- ☐ I used a meaningful commit message.

Activity K — Push the Project to GitHub

Your commit currently exists in your local repository.

Now synchronize or **push** the changes to GitHub.

After the push completes, open your GitHub repository in a browser.

Locate:

```
workshop-0/
```

Verify that your files are visible.

You've now completed:

```
VS CODE  
↓  
EDIT
```

```
↓  
GIT COMMIT  
↓  
PUSH  
↓  
GITHUB
```

Verification Checkpoint K

- ☐ My commit was pushed successfully.
- ☐ I can see the **workshop-0** files on GitHub.
- ☐ I can locate my commit history.
- ☐ I understand that GitHub now contains a remote copy of the committed work.

Activity L — Publish with GitHub Pages

Now you'll make web content from the repository available through the Web.

Configure **GitHub Pages** according to the procedure demonstrated by your instructor.

Your instructor will explain:

- the publishing source;
- repository configuration;
- the generated web address;
- any limitations associated with the repository structure being used in the course.

Once GitHub Pages is available, open your published website.

Your URL will be different from the Live Server address.

Compare the Two Environments

Local Development

```
VS CODE  
↓  
LIVE SERVER  
↓  
localhost / 127.0.0.1
```

This is where you develop and test your website on your own computer.

Published Website

```
VS CODE
  ↓
GIT
  ↓
GITHUB
  ↓
GITHUB PAGES
  ↓
PUBLIC WEB ADDRESS
```

The published website can now be requested from another browser over the Internet.

Verification Checkpoint L

- ☐ GitHub Pages is configured.
- ☐ I can locate my published web address.
- ☐ My published webpage loads.
- ☐ I understand the difference between Live Server and GitHub Pages.
- ☐ I understand that published content may be visible outside NSCC.

Activity M — Make a Change and Follow the Complete Workflow

You've configured all of the individual pieces. Now you'll put them together.

Modify your Workshop 0 webpage.

Add a new section inside `<main>`:

```
<section>
  <h2>My Development Workflow</h2>

  <p>
    I can develop a website locally, track my changes
    using Git, store my project on GitHub, and publish
    web content using GitHub Pages.
  </p>
</section>
```

Save the document.

Verify the change using Live Server.

Inspect it using Developer Tools.

Commit the change using Git.

Use a meaningful commit message such as:

```
Add development workflow section
```

Push the commit to GitHub.

Wait for GitHub Pages to update if necessary.

Open your published website.

Verify that the new section appears.

You've now performed the complete WEBD 1000 workflow:

```
PLAN
↓
EDIT IN VS CODE
↓
SAVE
↓
VIEW WITH LIVE SERVER
↓
INSPECT / TEST
↓
COMMIT WITH GIT
↓
PUSH TO GITHUB
↓
PUBLISH WITH GITHUB PAGES
↓
VERIFY THE PUBLIC WEBSITE
```

Verification Checkpoint M

- ☐ I modified my project locally.
- ☐ I tested the modification locally.
- ☐ I inspected the page.
- ☐ I committed the change.
- ☐ I pushed the change.
- ☐ GitHub contains the new commit.
- ☐ the published website contains the new content.

Reflection

Answer the following questions in your own words.

1. VS Code

What role does VS Code play in your web-development environment?

2. Live Server

Why might a developer use Live Server instead of repeatedly opening HTML files manually?

3. Git

What problem does Git solve?

4. GitHub

What is the difference between Git and GitHub?

5. GitHub Pages

What is the difference between a GitHub repository and a website published using GitHub Pages?

6. Local vs. Published

What is the difference between viewing your website using Live Server and viewing it through GitHub Pages?

7. Development History

Why might meaningful Git commits be useful when working on a semester-long project?

8. Public Content

What types of information should **not** be placed in a public GitHub repository or GitHub Pages website?

9. Looking Forward

What part of the web-development workflow do you think will require the most practice?

Deliverables

Workshop 0 is complete when you have the following.

Development Environment

- Working VS Code installation
- Working Git installation
- Live Server installed
- Access to browser Developer Tools

GitHub

- Working GitHub account

- WEBD1000 repository
- Local copy of the repository
- At least two meaningful Git commits
- Project successfully pushed to GitHub

Website

Your repository contains:

```
WEBD1000/  
├── workshop-0/  
│   ├── index.html  
│   ├── css/  
│   │   └── style.css  
│   └── images/
```

Publishing

- GitHub Pages configured as directed
- Published webpage successfully accessible
- Published page reflects the latest committed version

Reflection

- Completed Workshop 0 reflection questions

Submission

Workshop 0 is formative and required.

Follow your instructor's directions for confirming completion.

Be prepared to provide:

1. the URL of your GitHub repository;
2. the URL of your published GitHub Pages site;
3. evidence of your Git commit history;
4. your completed Workshop 0 reflection;
5. a demonstration of your local VS Code and Live Server environment if requested.

Do not submit passwords, authentication tokens, recovery codes, or other account credentials.

Evaluation Criteria

Workshop 0 is evaluated as a **readiness check** rather than a graded technical assessment.

Readiness Area	Complete	Needs Attention
VS Code installed and operational	☐	☐
Git installed and operational	☐	☐
Live Server installed and operational	☐	☐
GitHub account accessible	☐	☐
WEBD1000 repository created	☐	☐
Repository available locally in VS Code	☐	☐
Workshop website runs locally	☐	☐
Browser Developer Tools can be used to inspect the page	☐	☐
Meaningful Git commits created	☐	☐
Changes successfully pushed to GitHub	☐	☐
GitHub Pages website successfully published	☐	☐
Student can explain the purpose of the major technologies	☐	☐
Reflection completed	☐	☐

Overall Status: ☐ Ready ☐ Requires Follow-Up

The objective is for every student to reach **Ready** before the development environment becomes a dependency for subsequent course activities.

Resources

Use the course resources provided by your instructor for current installation and configuration instructions.

Useful documentation includes:

- Visual Studio Code documentation
- Git documentation
- GitHub documentation
- GitHub Pages documentation
- Live Server extension documentation
- Browser Developer Tools documentation
- WEBD 1000 Brightspace resources
- WEBD 1000 Microsoft Teams channel

When installing development tools or extensions, use trusted sources and verify the publisher before installing software.

Academic Integrity

The purpose of this workshop is to establish **your own development environment** and confirm that you understand how it works.

You may work alongside classmates and help one another troubleshoot installation or configuration problems. However, you should perform the configuration on your own account and development environment.

Do not:

- share passwords or authentication credentials;
- use another student's GitHub account;
- submit another student's repository as your own;
- copy another student's reflection responses;
- expose private credentials in source code or repositories.

When using AI tools or other external resources for troubleshooting, you remain responsible for understanding and verifying the commands and configuration changes you apply to your computer.

Copyright Notice

Course materials, examples, starter files, images, and other resources provided by your instructor are intended for educational use within WEBD 1000 unless otherwise indicated.

Do not assume that course materials or third-party content may be publicly redistributed simply because GitHub allows a repository to be made public.

When developing websites throughout this course, use content that you have created, content you have permission to use, or appropriately licensed resources, and provide attribution where required.

Instructor Note

Workshop 0 establishes the technical baseline for the remainder of WEBD 1000.

Students aren't expected to demonstrate mastery of HTML, Git, GitHub, or web hosting at this stage. The emphasis is on establishing a functioning workflow and developing an introductory mental model of how the technologies fit together.

The workshop intentionally establishes **GitHub Pages early in the semester**. This allows the Semester Website Project to develop as an externally viewable web project rather than existing solely as an LMS submission or collection of local files.

The workflow introduced here should therefore become normal practice throughout the course:

Develop → Test → Commit → Push → Publish → Verify

Progressing to the Next Stage

You now have the tools required to **create, inspect, manage, and publish web content**.

That sets up the next question in the course.

It's no longer simply:

Can we make a webpage appear in a browser?

We can now ask:

What makes that webpage effective for the people who need to use it?

In **Workshop 1 — Website Evaluation**, you'll move much more directly into **Outcome 1** by evaluating websites for:

- usability;
- accessibility;
- contrast;
- repetition;
- alignment;
- proximity;
- navigation;
- readability;
- overall user experience.

The course progression has begun:

Tooling & Workflow Readiness → Website Evaluation → Website Planning → HTML Development → CSS & Visual Design → Responsive Development → Quality Assurance → Final Website